

Documento Técnico do Projeto

Pipeline ETL/ELT de Despesas Parlamentares no GCP

Python • Apache Airflow • Docker • Google Cloud Storage • BigQuery • SQL Analítico

Projeto	ETL Despesas Parlamentares no GCP
Autor	Luis Almeida
Perfil técnico	Engenharia de Dados Python GCP Airflow BigQuery
Status	Pipeline concluída; dashboard tratado como evolução opcional
Data	17/05/2026

Sumário executivo

Este documento apresenta a documentação técnica completa de uma pipeline de Engenharia de Dados construída para processar despesas parlamentares de deputados federais a partir da API Dados Abertos da Câmara dos Deputados. O projeto cobre a jornada de dados desde a extração da API até a disponibilização de tabelas e views analíticas no BigQuery, com orquestração via Apache Airflow em Docker.

A solução implementa uma arquitetura em camadas - Bronze, Silver e Gold - e foi construída com foco em reprodutibilidade, organização profissional, separação de responsabilidades, controle de qualidade, integração cloud e documentação de portfólio.

O dashboard foi deliberadamente deixado fora do escopo final da pipeline. As views analíticas já foram criadas e estão preparadas para conexão futura com Looker Studio ou Power BI.

Resultado final

Bloco	Resultado entregue
Extração	Coleta de deputados e despesas via API pública da Câmara
Bronze	Dados brutos em JSON organizados por entidade e data de carga
Silver	Dados tratados em Parquet com padronização, conversão de tipos e deduplicação
Quality checks	Validações automáticas antes da carga cloud
GCS	Data Lake cloud com camadas Bronze e Silver
BigQuery	Staging, modelo Gold e views analíticas
Airflow	Pipeline orquestrada em DAG local com Docker
Documentação	README, arquitetura, dicionário de dados, guia de execução e evidências visuais

Storytelling do projeto

O projeto nasceu a partir de um objetivo claro: construir uma pipeline de dados realista, com características próximas às práticas de Engenharia de Dados usadas em ambientes profissionais, mas mantendo controle de custo e execução local sempre que possível.

A fonte escolhida foi a API Dados Abertos da Câmara dos Deputados, que disponibiliza informações públicas sobre deputados e despesas parlamentares. O dado público tem valor técnico porque apresenta desafios reais: paginação, instabilidade temporária de API, campos nulos, duplicidades, valores negativos, diversidade de tipos e necessidade de modelagem analítica.

A primeira etapa foi organizar o projeto como um repositório profissional, separando código-fonte, dados, SQL, documentação, DAGs, testes e configuração. Em seguida, foi criada a extração, inicialmente simples, depois robustecida com logs, retries e parametrização por ano, mês e limite de deputados.

Depois da extração, os dados brutos foram preservados na camada Bronze. A partir deles, a pipeline criou uma camada Silver em Parquet, com colunas padronizadas, tipos tratados e deduplicação técnica. Antes de enviar qualquer dado para a nuvem, foram adicionados quality checks para evitar cargas vazias, inconsistentes ou estruturalmente incorretas.

Com a base local validada, a solução avançou para o Google Cloud Platform: criação de projeto, bucket no GCS, Service Account, upload das camadas e carga dos Parquets no BigQuery. No BigQuery, foi criada a camada Gold, composta por dimensões, fato e views analíticas. Por fim, o fluxo foi encapsulado em uma DAG no Airflow local com Docker, fechando a orquestração ponta a ponta.

A história do projeto, portanto, demonstra uma evolução gradual: de scripts locais para uma pipeline cloud orquestrada, documentada e reproduzível.

Objetivos

Objetivo geral

Construir uma pipeline de Engenharia de Dados para processar despesas parlamentares, estruturando o fluxo de dados desde a extração da API pública até a disponibilização de tabelas analíticas no BigQuery.

Objetivos específicos

- Consumir a API Dados Abertos da Câmara dos Deputados de forma parametrizável.
- Armazenar os dados brutos em uma camada Bronze local e cloud.
- Transformar os dados em uma camada Silver com formato Parquet.
- Aplicar validações de qualidade antes da carga cloud.
- Enviar dados Bronze e Silver para o Google Cloud Storage.
- Carregar Parquets da camada Silver no BigQuery.
- Criar modelo analítico Gold com dimensões e tabela fato.
- Criar views analíticas para consumo futuro em dashboards.
- Orquestrar a pipeline com Apache Airflow em Docker.
- Documentar o projeto para uso como portfólio de Engenharia de Dados.

Escopo final

Incluído no escopo	Fora do escopo final
Pipeline ETL/ELT completa	Dashboard no Looker Studio
GCS e BigQuery	Cloud Composer
Airflow local com Docker	Processamento contínuo em produção
Documentação técnica	CI/CD com GitHub Actions
Views analíticas	Carga histórica completa de todos os anos

Fonte de dados

A fonte de dados utilizada foi a API Dados Abertos da Câmara dos Deputados. Foram usados principalmente dois tipos de endpoint: listagem de deputados e despesas por deputado.

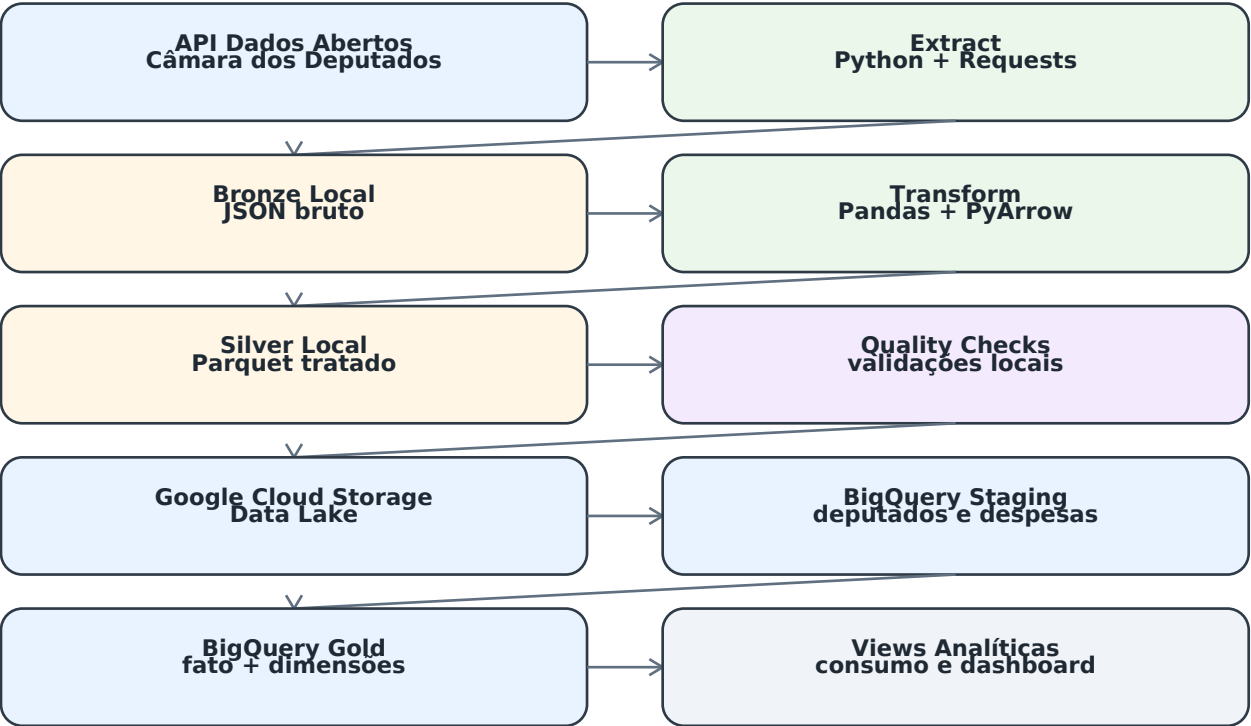
Endpoint	Finalidade
/deputados	Obter a lista de deputados federais ativos no recorte de consulta

Endpoint	Finalidade
/deputados/{id}/despesas	Obter despesas parlamentares por deputado, ano e mês

O recorte validado no MVP processou 513 deputados na dimensão cadastral e 5 deputados na extração de despesas do mês 03/2026, resultando em 146 despesas tratadas após deduplicação técnica.

Arquitetura da solução

A arquitetura foi organizada em camadas para separar responsabilidades e permitir rastreabilidade, reprocessamento e consumo analítico.



Fluxo lógico

```
API Dados Abertos Câmara
↓
Extração com Python
↓
Bronze Local - JSON bruto
↓
Transformação com Pandas
↓
Silver Local - Parquet tratado
↓
Quality Checks
↓
Google Cloud Storage
↓
```

BigQuery Staging
↓
BigQuery Gold
↓
Views Analíticas
↓
Dashboard / Análise futura

Stack técnica

Camada	Tecnologias
Linguagem	Python 3.12
Extração	requests
Tratamento	pandas, numpy, pyarrow
Validação	pandas e scripts próprios
Data Lake	Google Cloud Storage
Data Warehouse	BigQuery
Orquestração	Apache Airflow
Containerização	Docker e Docker Compose
Configuração	python-dotenv
Versionamento	Git e GitHub
Ambiente	WSL Ubuntu

Estrutura do repositório

```
etl-despesas-parlamentares-gcp/  
├── dags/  
│   └── camara_deputados_etl_dag.py  
├── data/  
│   ├── raw/bronze/  
│   ├── processed/silver/  
│   └── output/  
├── docs/  
│   ├── architecture.md  
│   ├── data_dictionary.md  
│   ├── execution_guide.md  
│   └── images/  
├── sql/  
│   ├── create_tables.sql  
│   ├── create_views.sql  
│   └── analytical_queries.sql  
├── src/  
│   ├── extract/  
│   ├── transform/  
│   ├── quality/  
│   ├── load/  
│   └── utils/  
├── Dockerfile.airflow  
├── docker-compose.yaml  
├── requirements.txt  
├── requirements-airflow.txt  
├── .env.example  
└── README.md
```

Camadas de dados

Camada Bronze

A camada Bronze preserva o dado bruto conforme retornado pela API, sem transformação estrutural. Essa camada permite auditoria, reprocessamento e comparação com a origem.

```
data/raw/bronze/deputados/dt_carga=YYYY-MM-DD/deputados.json
data/raw/bronze/despesas/ano=YYYY/mes=MM/dt_carga=YYYY-MM-DD/despesas.json

gs://etl-camara-deputados-luis-2026-data-lake/bronze/
```

Camada Silver

A camada Silver armazena dados tratados em Parquet. Nessa etapa ocorrem padronização de nomes, conversão de tipos, deduplicação técnica e inclusão de metadados de carga.

```
data/processed/silver/deputados/deputados.parquet
data/processed/silver/despesas/ano=2026/mes=03/despesas.parquet

gs://etl-camara-deputados-luis-2026-data-lake/silver/
```

Camada Gold

A camada Gold é materializada no BigQuery, com tabelas dimensionais, tabela fato e views analíticas. Essa camada representa o dado pronto para consulta e visualização.

```
dim_deputado
dim_partido
dim_fornecedor
dim_tipo_despesa
fato_despesas

vw_kpis_gerais
vw_ranking_deputados
vw_ranking_partidos
vw_ranking_ufs
vw_ranking_fornecedores
vw_ranking_tipos_despesa
vw_evolucao_mensal
```

Processo da pipeline

1. Extração

A extração coleta dados de deputados e despesas parlamentares. A implementação considera paginação, parâmetros de ano/mês, limite de deputados e retries para falhas temporárias da API.

2. Transformação

A transformação lê os arquivos JSON da camada Bronze e gera arquivos Parquet na camada Silver. Os dados são padronizados, convertidos e deduplicados.

3. Quality checks

Os quality checks impedem que dados vazios, estruturalmente inválidos ou duplicados avancem para a nuvem. Valores negativos em campos monetários são tratados como warning, pois podem representar ajustes, estornos ou devoluções.

4. Upload para GCS

As camadas Bronze e Silver são enviadas para o bucket no Google Cloud Storage, mantendo a mesma organização lógica usada localmente.

5. Carga no BigQuery

Os arquivos Parquet da camada Silver são carregados no BigQuery como tabelas staging: deputados e despesas.

6. Camada Gold

Scripts SQL criam dimensões, tabela fato e views analíticas. Essa etapa é automatizada por script Python e executada também pela DAG do Airflow.

7. Orquestração

O Airflow executa a sequência `extract`, `transform`, `quality_checks`, `upload_gcs_and_load_bigquery` e `run_gold_sql`. A execução foi validada com todas as tasks em sucesso.

Modelo analítico no BigQuery

O modelo Gold segue uma abordagem dimensional simples, adequada para análise exploratória e dashboards.

Tabela/View	Tipo	Função
dim_deputado	Dimensão	Informações cadastrais dos deputados
dim_partido	Dimensão	Partidos políticos derivados dos deputados
dim_fornecedor	Dimensão	Fornecedores presentes nas despesas
dim_tipo_despesa	Dimensão	Categorias de despesas
fato_despesas	Fato	Eventos financeiros de despesas parlamentares
vw_kpis_gerais	View	Indicadores consolidados
vw_ranking_deputados	View	Ranking por deputado
vw_ranking_partidos	View	Ranking por partido
vw_ranking_ufs	View	Ranking por unidade federativa
vw_ranking_fornecedores	View	Ranking por fornecedor
vw_ranking_tipos_despesa	View	Ranking por categoria
vw_evolucao_mensal	View	Série temporal mensal

Resultados validados

O recorte final validado processou dados de março de 2026 para cinco deputados na etapa de despesas, mantendo a dimensão completa de deputados.

Métrica	Valor
Deputados carregados	513

Métrica	Valor
Despesas brutas extraídas no recorte	163
Despesas tratadas após deduplicação	146
Deputados com despesas no recorte	5
Fornecedores únicos	84
Tipos de despesa	11
Total valor documento	R\$ 251.384,89
Total valor líquido	R\$ 251.253,99
Total valor glosa	R\$ 130,90
Média valor líquido	R\$ 1.720,92

Orquestração com Airflow

A orquestração foi implementada com Apache Airflow local em Docker. A decisão evitou o custo de Cloud Composer e manteve a prática de orquestração profissional.

```
camara_deputados_etl_dag
├── extract
│   └── ↓
│       ├── transform
│       │   └── ↓
│       │       ├── quality_checks
│       │       │   └── ↓
│       │       │       ├── upload_gcs_and_load_bigquery
│       │       │       │   └── ↓
│       │       │       └── run_gold_sql
```

Durante a validação, foram resolvidos problemas típicos de ambiente Docker/WSL, como permissões de escrita em pastas montadas e permissão de leitura da chave da Service Account dentro do container.

Segurança, custos e operação

Segurança

- Arquivo .env não versionado.
- Pasta credentials/ ignorada pelo Git.
- Chave JSON da Service Account mantida apenas localmente.
- Service Account usada para autenticação com GCS e BigQuery.
- Bucket configurado com uniform bucket-level access.

Controle de custo

- Cloud Composer, VMs, Dataflow e Cloud SQL ficaram fora do MVP.
- Airflow foi executado localmente com Docker.
- Bucket criado como regional, classe STANDARD, sem versionamento e sem soft delete.
- Após conclusão, o faturamento do GCP foi desativado para evitar cobranças futuras.

Operação

```
# Executar pipeline local
python -m src.run_pipeline --ano 2026 --mes 3 --limite-deputados 5

# Subir Airflow local
docker compose --env-file .env.airflow up -d

# Parar Airflow local
docker compose --env-file .env.airflow down
```

Documentação entregue

Arquivo	Conteúdo
README.md	Visão geral, stack, arquitetura, execução e status
docs/architecture.md	Arquitetura técnica e decisões do projeto
docs/data_dictionary.md	Descrição das tabelas, campos e views
docs/execution_guide.md	Guia operacional de execução local, cloud e Airflow
docs/images/	Evidências visuais do projeto

Conclusão

A pipeline foi concluída com sucesso como um projeto de Engenharia de Dados ponta a ponta. O fluxo entrega ingestão, transformação, validação, armazenamento cloud, modelagem analítica e orquestração. O projeto também está documentado e versionado, pronto para apresentação em portfólio.

O dashboard foi tratado como evolução futura e não como dependência da pipeline. As views analíticas já criadas no BigQuery deixam a camada de consumo preparada para Looker Studio ou Power BI.

Resumo da entrega final

```
API Câmara
↓
Python Extract
↓
Bronze JSON
↓
Pandas Transform
↓
Silver Parquet
↓
Quality Checks
↓
Google Cloud Storage
↓
BigQuery Staging
↓
BigQuery Gold
↓
Views Analíticas
↓
Airflow + Docker
↓
Documentação e GitHub
```

Evoluções futuras

Evolução	Prioridade
Criar dashboard no Looker Studio	Alta, mas opcional
Processar todos os deputados	Alta
Processar múltiplos meses e anos	Alta
Adicionar testes com pytest	Média
Particionar e clusterizar tabelas BigQuery	Média
Criar GitHub Actions para validação	Média
Criar diagrama visual da arquitetura	Média